



Exercícios : Interação entre tarefas

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Baseado no conteúdo do livro “Sistemas Operacionais: Conceitos e Mecanismos” do Prof. Carlos A. Maziero (UFPR)

Capítulos 8 a 13

1. Quais são as vantagens e desvantagens das abordagens a seguir, sob as óticas do sistema operacional e do programador de aplicativos?
 - (a) comunicação bloqueante ou não-bloqueante
 - (b) canais com buffering ou sem buffering
 - (c) comunicação por mensagens ou por fluxo
 - (d) mensagens de tamanho fixo ou variável
 - (e) comunicação 1:1 ou M:N
2. Explique como processos que comunicam por troca de mensagens se comportam em relação à capacidade do canal de comunicação, considerando as semânticas de chamada síncrona e assíncrona.
3. Sobre as afirmações a seguir, relativas mecanismos de comunicação, indique quais são incorretas, justificando sua resposta:
 - (a) A comunicação indireta (por canais) é mais adequada para sistemas distribuídos.
 - (b) Canais com capacidade finita somente são usados na definição de algoritmos, não sendo implementáveis na prática.
 - (c) Na comunicação direta, o emissor envia os dados diretamente a um canal de comunicação.
 - (d) Na comunicação por fluxo, a ordem dos dados enviados pelo emissor é mantida do lado receptor.
 - (e) Na comunicação por troca de mensagens, o núcleo transfere pacotes de dados do processo emissor para o processo receptor.
4. Sobre as afirmações a seguir, relativas à sincronização na comunicação entre processos, indique quais são incorretas, justificando sua resposta:
 - (a) Na comunicação semi-bloqueante, o emissor espera indefinidamente pela possibilidade de enviar os dados.
 - (b) Na comunicação síncrona ou bloqueante, o receptor espera até receber a mensagem.
 - (c) Um mecanismo de comunicação semi-bloqueante com prazo $t = \infty$ equivale a um mecanismo bloqueante.
 - (d) Na comunicação síncrona ou bloqueante, o emissor retorna uma mensagem de erro caso o receptor não esteja pronto para receber a mensagem.
 - (e) Se o canal de comunicação tiver capacidade nula, emissor e receptor devem usar mecanismos não-bloqueantes.

- (f) A comunicação não-bloqueante em ambos os participantes só é viável usando canais de comunicação com buffer não-nulo.
5. Classifique as filas de mensagens UNIX de acordo com os tipos de comunicação discutidos.
6. Classifique os pipes UNIX de acordo com os tipos de comunicação discutidos.
7. Classifique as áreas de memória compartilhadas de acordo com os tipos de comunicação discutidos.
8. Sobre as afirmações a seguir, relativas aos mecanismos de comunicação, indique quais são incorretas, justificando sua resposta:
- (a) As filas de mensagens POSIX são um exemplo de canal de comunicação com capacidade nula.
 - (b) A memória compartilhada provê mecanismos de sincronização para facilitar a comunicação entre os processos.
 - (c) A troca de dados através de memória compartilhada é mais adequada para a comunicação em rede.
 - (d) Processos que se comunicam por memória compartilhada podem acessar a mesma área da RAM.
 - (e) Os pipes Unix são um bom exemplo de comunicação M:N.
 - (f) A comunicação através de memória compartilhada é particularmente indicada para compartilhar grandes volumes de dados entre dois ou mais processos.
 - (g) As filas de mensagens POSIX são um bom exemplo de canal de eventos.
 - (h) Nas filas de mensagens POSIX, as mensagens transitam através de arquivos em disco criados especialmente para essa finalidade.
 - (i) Em UNIX, um pipe é um canal de comunicação unidirecional que liga a saída padrão de um processo à entrada padrão de outro.
9. Explique o que são condições de disputa, mostrando um exemplo real.
10. O que são seções críticas e o termo exclusão mútua?
11. Sobre as afirmações a seguir, relativas aos mecanismos de coordenação, indique quais são incorretas, justificando sua resposta:
- (a) A estratégia de inibir interrupções para evitar condições de disputa funciona em sistemas multi-processados.
 - (b) Os mecanismos de controle de entrada nas regiões críticas provêem exclusão mútua no acesso às mesmas.
 - (c) Os algoritmos de busy-wait se baseiam no teste contínuo de uma condição.
 - (d) Condições de disputa ocorrem devido às diferenças de velocidade na execução dos processos.
 - (e) Condições de disputa ocorrem quando dois processos tentam executar o mesmo código ao mesmo tempo.
 - (f) O algoritmo de Peterson garante justiça no acesso à região crítica.
 - (g) Os algoritmos com estratégia busy-wait otimizam o uso da CPU do sistema.
 - (h) Uma forma eficiente de resolver os problemas de condição de disputa é introduzir pequenos atrasos nos processos envolvidos.

12. Explique o que é espera ocupada e por que os mecanismos que empregam essa técnica são considerados ineficientes.
13. Considere ocupado uma variável inteira compartilhada entre dois processos A e B (inicialmente, ocupado = 0). Sendo que ambos os processos executam o trecho de programa abaixo, explique em que situação A e B poderiam entrar simultaneamente nas suas respectivas regiões críticas.

```
while (true) {
    regioao_ao_critica();
    while (ocupado) {};
    ocupado = 1;
    regioao_critica();
    ocupado = 0;
}
```

14. Explique como funciona os mecanismos de coordenação semáforos, mutex e monitores.
15. Em que situações um semáforo deve ser inicializado em 0, 1 ou $n > 1$?
16. Desenhe o diagrama de tempo da execução e indique as possíveis saídas para a execução concorrente das duas threads cujos pseudo-códigos são descritos a seguir. Os semáforos s_1 e s_2 estão inicializados com zero (0).

<pre>thread1 () { down (s1) ; printf ("A") ; up (s2) ; printf ("B") ; }</pre>	<pre>thread2 () { printf ("X") ; up (s1) ; down (s2) ; printf ("Y") ; }</pre>
---	---

17. Explique o problema dos produtores/consumidores e uma possível solução para coordenação das tarefas.
18. Explique o problema dos leitores/escritores e uma possível solução para coordenação das tarefas.
19. Explique o problema do jantar dos filósofos e uma possível solução para coordenação das tarefas.
20. Explique cada uma das quatro condições necessárias para a ocorrência de impasses.
21. Uma vez detectado um impasse, quais as abordagens possíveis para resolvê-lo? Explique-as e comente sua viabilidade.
22. O trecho de código a seguir apresenta uma solução para o problema do jantar dos filósofos, mas essa solução apresenta risco de impasse. Explique por que e como podem ocorrer impasses nessa solução.

```
#define N 5
sem_t garfo[5] ; // 5 semáforos iniciados em 1
void filosofo (int i){
    while (1) {
        medita () ;
        sem_down (garfo [i]) ;
        sem_down (garfo [(i+1) % N]) ;
        come () ;
        sem_up (garfo [i]) ;
        sem_up (garfo [(i+1) % N]) ;
    }
}
```

23. Sobre as afirmações a seguir, relativas a impasses, indique quais são incorretas, justificando sua resposta:

- (a) Impasses ocorrem porque vários processos tentam usar o processador ao mesmo tempo.
- (b) Os sistemas operacionais atuais provêem vários recursos de baixo nível para o tratamento de impasses.
- (c) Podemos encontrar impasses em sistemas de processos que interagem unicamente por mensagens.
- (d) As condições necessárias para a ocorrência de impasses são também suficientes se houver somente um recurso de cada tipo no conjunto de processos considerado.